

Sequence → physicochemical scales → interpretable features → explainable ML → biological mechanism

`pip install aaanalysis · aaanalysis.readthedocs.io`

AAanalysis is a Python framework for interpretable, sequence-based protein prediction. It turns sequences into physicochemical features (CPP), trains explainable models, and traces every prediction back to a residue × property × group comparison — robust for small datasets.

The Golden Workflow

canonical pipeline

- 1 **LOAD** `load_dataset · load_scales → df_seq · df_scales`
- 2 **PARTS** `SequenceFeature.get_df_parts → df_parts`
- 3 **FEATURES** `CPP.run → df_feat`
- 4 **MODEL** `TreeModel.fit · dPULearn.fit → feat_importance · labels_`
- 5 **EXPLAIN** `CPPPlot.feature_map · ShapModel → figure · feat_impact`

Prediction Task Levels

task → setup

Residue AA_* positional
unit: sliding window (aa_window_size)
ref: non-site windows / shuffled background

Domain DOM_* both
unit: part-set jmd_n · tmd · jmd_c (from tmd_start / tmd_stop)
ref: labelled A vs B groups

Protein SEQ_* compositional
unit: whole chain (composition)
ref: labelled groups / composition-matched background

Sequence Anatomy

the TMD model

TMD Target Middle Domain — the central segment of interest (e.g. transmembrane stretch, binding region).

JMD Juxta Middle Domain — the flanks adjoining the TMD (jmd_n on the N-side, jmd_c on the C-side).



1: positives 0: rel-negatives 2: unlabeled

Feature Ontology

Part × Split × Scale

PART where on the sequence

tmd · jmd_n · jmd_c · tmd_jmd · jmd_n_tmd_n · tmd_c_jmd_c

SPLIT how to read the part

Segment — contiguous · Pattern — sparse pairs · PeriodicPattern — i, i+3/4

SCALE which physicochemical property

AAontology (~600 scales) · hydrophobicity · charge · helix propensity · shape

TMD × Segment × hydrophobicity → membrane insertion
JMD × Pattern × net charge → electrostatic recognition
TMD × PeriodicPattern × helix → α-helical interface

AAanalysis · interpretable, sequence-based protein prediction

CPP Strategies

via split_kws

Compositional = sequence/protein-level

one whole-part average (composition-like, position-agnostic)

```
skw = sf.get_split_kws(
    split_types="Segment",
    n_split_min=1, n_split_max=1)
cpp = aa.CPP(df_parts=df_parts,
             split_kws=skw)
```

Positional = residue-/region-level

sub-segments and/or patterns resolved to positions

```
skw = sf.get_split_kws(
    split_types=["Segment", "Pattern",
                 "PeriodicPattern"],
    n_split_max=5, steps_pattern=[3, 4],
    steps_periodicpattern=[3, 4])
cpp = aa.CPP(df_parts=df_parts,
             split_kws=skw)
```

Domain level uses both. → CPP strategies: see the CPP tutorial (docs).

Which Module Should I Use?

intent → module

Explore sequence patterns / composition →

AAlogo

Discover discriminative physicochemical features →

CPP

Reduce redundant amino-acid scales →

AAclust

Train with positives + unlabelled data →

dPULearn

Train an interpretable classifier →

TreeModel

Explain a prediction (per feature / sample) →

ShapModel [pro]

Visualize CPP features →

CPPPlot

Data & Preparation

load · encode · clean

Load benchmark sequences

`load_dataset(name) → df_seq`

Load AAontology scales

`load_scales() → df_scales`

Load precomputed features

`load_features(name) → df_feat`

Read / write FASTA

`read_fasta(file) → df_seq`

Encode sequences (one-hot / int)

`SequencePreprocessor().encode_*(seqs)`

Cluster redundant homologs

`filter_seq(df_seq)` [pro]

Feature Engineering

parts · CPP · scales

Extract sequence parts

`get_df_parts(df_seq) → df_parts`

Discover discriminative features

`CPP(df_parts).run(labels) → df_feat *`

Simplify → interpretable scales

`CPP.simplify(df_feat, labels)`

Build feature matrix X

`feature_matrix(df_feat, df_parts) → X`

Reduce redundant scales

`AAclust().fit(X)` [Wrapper]

Drop correlated features

`filter_correlation(X)`

Structural & Embedding

alternative feature sources

Fetch AlphaFold + encode 3D/DSSP/PAE

`StructurePreprocessor().encode_dssp(df_seq)`

Encode protein-LM embeddings

`EmbeddingPreprocessor().encode(df_seq)`

Embeddings → pseudo-scales

`EmbeddingPreprocessor().build_scales(df_seq)`

Combine numeric feature dicts → CPP

`combine_dict_nums(dict_nums)`

Modeling & Explainability

Train + RFE + MC importance

`TreeModel().fit(X, labels)` [Wrapper]

Train with positives + unlabelled

`dPULearn().fit(X, labels)` [Wrapper]

Per-feature / sample SHAP impact

`ShapModel().fit(X, labels)` [pro]

Sequence Analysis

logos · motifs

Position-specific logo

`AAlogo().get_df_logo(df_parts)`

Sample sequence windows

`AAWindowSampler().sample(df_seq)`

Pairwise sequence similarity

`comp_seq_sim(df_seq)` [pro]

Scan motifs (FIMO / MEME)

`scan_motif(df_seq, pwm)` [pro]

Metrics & Plotting

utilities

Adjusted AUC (class imbalance)

`comp_auc_adjusted(X, labels)`

BIC score · KL divergence

`comp_bic_score(X, labels) · comp_kld`

Global plot style & fonts

`plot_settings(font_scale)`

Colours & standalone legend

`plot_get_clist(n) · plot_legend(ax)`

Protein Design (planned)

In-silico point mutations

AAmut · **AAmutPlot**

Sequence-design libraries

SeqMut · **SeqMutPlot**

Install · Import · Load

core + [pro]

```
# Python >= 3.11
pip install aaanalysis # core
pip install 'aaanalysis[pro]' # SHAP, FIMO, Bio

import numpy as np
import matplotlib.pyplot as plt
import aaanalysis as aa

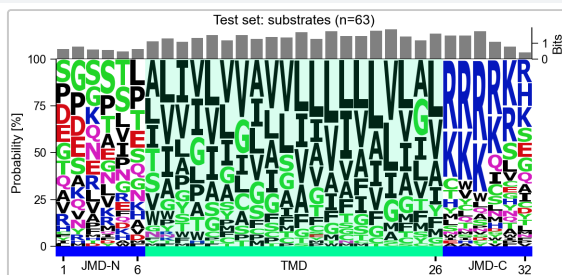
df_seq = aa.load_dataset(name='DOM_GSEC') # γ-secretase
labels = df_seq['label'].to_list()
df_scales = aa.load_scales()
# TMD model: 20-aa TMD, short JMD flanks
aa.options['jmd_n_len'] = 6
aa.options['jmd_c_len'] = 6
```

AAlogo — see the data

dataset at a glance

```
sf = aa.SequenceFeature()
df_parts = sf.get_df_parts(df_seq=df_seq,
    list_parts=['tmd', 'jmd_n', 'jmd_c'])

aa.plot_settings(font_scale=0.7)
# aal kws builds df_logo + bits bar for you
aa.AAlogoPlot().single_logo(
    aal_kws=dict(df_parts=df_parts, labels=labels,
        label_test=1, tmd_len=20),
    name_data='Test set: substrates')
plt.tight_layout(); plt.show()
```

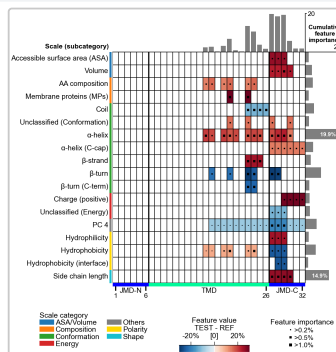


C++ — feature map

flagship · interpretability

```
# extended parts -> default split grid applies
df_parts = sf.get_df_parts(df_seq=df_seq,
    list_parts=['tmd', 'jmd_n', 'jmd_c'])
cpp = aa.CPP(df_parts=df_parts, df_scales=df_scales)
df_feat = cpp.run(labels=labels, n_filter=40)
# swap scales for more interpretable correlated ones
df_feat = cpp.simplify(df_feat=df_feat, labels=labels)
X = sf.feature_matrix(features=df_feat['feature'],
    df_parts=df_parts)
tm = aa.TreeModel(); tm.fit(X, labels=labels)
df_feat = tm.add_feat_importance(df_feat=df_feat)

cpp_plot = aa.CPPPlot()
aa.plot_settings(font_scale=0.65)
cpp_plot.feature_map(df_feat=df_feat)
plt.tight_layout(); plt.show()
```

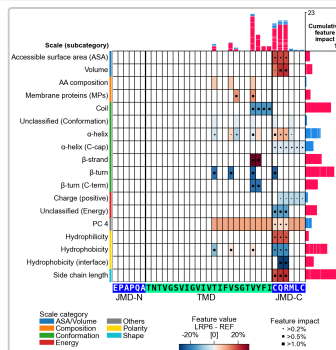


ShapModel — explain a prediction

per-protein · [pro]

```
se = aa.ShapModel()
# fuzzy_labeling captures the true SHAP impact
se.fit(X, labels=labels, fuzzy_labeling=True)
# explain a borderline call — LRP6 (~60% substrate)
i = list(df_seq['entry']).index('075581')
df_feat = se.add_feat_impact(df_feat=df_feat,
    sample_positions=i, names='LRP6')

cpp_plot.feature_map(df_feat=df_feat, shap_plot=True,
    col_imp='feat_impact_LRP6', name_test='LRP6',
    **args_seq)
plt.tight_layout(); plt.show()
```

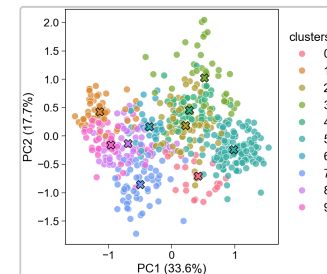


AAclust — clusters

scale reduction · Wrapper

```
X = np.array(df_scales).T
aac = aa.AAclust()
aac.fit(X, names=list(df_scales), n_clusters=10)
aac.medoid_names_ # redundancy-reduced scales

aac_plot = aa.AAclustPlot()
aac_plot.centers(X, labels=aac.labels_)
plt.tight_layout(); plt.show()
```

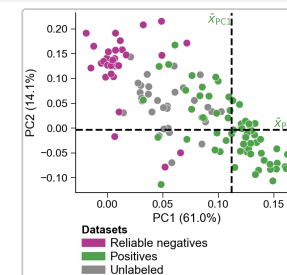


dpULearn — PCA

reliable negatives · Wrapper

```
# labels: 1 = positive, 2 = unlabelled
dpul = aa.dpULearn()
dpul.fit(X=X, labels=labels, n_unl_to_neg=n_pos)
df_pu = dpul.df_pu_ # 1 pos · 0 rel-neg · 2 unl

dpul_plot = aa.dpULearnPlot()
dpul_plot.pca(df_pu=df_pu, labels=labels)
plt.tight_layout(); plt.show()
```



The 5-Minute Example

copy-paste this

```
import matplotlib.pyplot as plt
import aaanalysis as aa
df_seq = aa.load_dataset(name='DOM_GSEC')
labels = df_seq['label'].to_list()
sf = aa.SequenceFeature()
df_parts = sf.get_df_parts(df_seq=df_seq)
cpp = aa.CPP(df_parts=df_parts)
df_feat = cpp.run(labels=labels)
X = sf.feature_matrix(features=df_feat['feature'],
                      df_parts=df_parts)
tm = aa.TreeModel(); tm.fit(X, labels=labels)
df_feat = tm.add_feat_importance(df_feat=df_feat)
cpp_plot = aa.CPPPlot()
aa.plot_settings()
cpp_plot.feature_map(df_feat=df_feat)
plt.tight_layout(); plt.show()
```

Glossary

- Prediction level** — Residue (AA_*) · Domain (DOM_*) · Protein (SEQ_*) — the unit a task predicts at; sets the dataset, the part, and the reference.
- Part** — Named segment used as feature input: tmd, jmd_n, jmd_c, tmd_jmd, jmd_n_tmd_n, tmd_c_jmd_c.
- Split** — How a scale is read across a part: Segment (contiguous), Pattern (sparse), PeriodicPattern (i, i+3/4).
- Scale** — AA → ℝ mapping. AAontology ships ~600 curated scales in two-level categories.
- Feature** — (Part × Split × Scale) — the atomic, residue-grounded, interpretable unit of CPP.

aa.options

system-level settings

```
aa.options['random_state'] = 42
aa.options['verbose'] = True
aa.options['allow_multiprocessing'] = True

# TMD model — JMD flank widths
aa.options['jmd_n_len'] = 10
aa.options['jmd_c_len'] = 10

# plot labels & system-level scales
aa.options['name_tmd'] = 'TMD'
aa.options['df_scales'] = my_scales
```

Class ↔ Plot Class

mirrored API

CPP	CPPPlot	
AAclust	AAclustPlot	Wrapper
AAlogo	AAlogoPlot	
dPULearn	dPULearnPlot	Wrapper
TreeModel	—	Wrapper
ShapModel [pro]	—	Wrapper
AAMut	AAMutPlot	planned
SeqMut	SeqMutPlot	planned
AAWindowSampler	—	
SequenceFeature	—	
NumericalFeature	—	
SequencePreprocessor	—	

- Compositional vs positional** — How split_kws resolves locality: a whole-part average (compositional) vs sub-region/position-resolved (positional).
- df_seq** — Sequence table: entry, sequence, label, TMD bounds (tmd_start, tmd_stop).
- df_parts** — Wide table — one column per part (tmd, jmd_n, jmd_c, ...).
- df_feat** — Ranked features: feature, abs_auc, mean_dif, p_val, positions, scale, category.
- Wrapper** — sklearn-style class — .fit / .predict / .eval, sets trailing *_ attributes after fit.

How to Cite

if you use AAanalysis

- AAclust**
Breimann & Frishman (2024a), AAclust: k-optimized clustering for selecting redundancy-reduced sets of amino acid scales
Bioinformatics Advances
[Breimann24a]
- AAontology**
Breimann et al. (2024b), AAontology: An ontology of amino acid scales for interpretable machine learning
Journal of Molecular Biology
[Breimann24b]
- CPP & dPULearn**
Breimann & Kamp et al. (2025), Charting γ-secretase substrates by explainable AI
Nature Communications
[Breimann25a]

Design Principles

- Explicit over implicit — DataFrames everywhere
- Wrappers (.fit / .predict / .eval) set trailing *_ attributes after fit
- Biological interpretability is first-class
- Small-data robust and reproducible (layered seeds)

- Plot class** — *Plot mirror of an analytical class — same arguments, visualization only.
- PU labels** — dPULearn input: 1 = positive, 2 = unlabelled. Output: 1 / 0 (reliable-negative) / 2.
- CPP** — Comparative Physicochemical Profiling — discovers ranked Part × Split × Scale features.
- AAontology** — Two-level scale taxonomy; CPP uses its categories to organize and rank features.